DIVINE WORD COLLEGE OF LEGAZPI
SCHOOL OF ENGINEERING AND COMPUTER STUDIES
Rm. 217, Second Floor, DWCL South Campus
Cor. Rizal & Fr. Bates St., Old Albay District, Legazpi City, Philippines 4500

Divine Word of College of Legazpi
School of Engineering and Computer Sciences
Department of Computer Sciences and Information Technology
A Y 2026-2027

PermitLink: Event-Driven Integration of the Permit Application, Treasury, and Notification Systems for LGU Business Permits

In partial fulfillment of the
requirements for

IT PC 313: System Integration and Architecture

Team Nexora

Alcantara, Hannah L.
Diaz III, Jose Arym C.
Dibarosan, Princess Haneen S.
Fegason, Jericho M.

November 2026

DIVINE WORD COLLEGE OF LEGAZPI
SCHOOL OF ENGINEERING AND COMPUTER STUDIES
Rm. 217, Second Floor, DWCL South Campus
Cor. Rizal & Fr. Bates St., Old Albay District, Legazpi City, Philippines 4500

Title Page

Project Title: PermitLink: Event-Driven Integration of the Permit Application, Treasury, and Notification Systems for LGU Business Permits

Approved Scenario: Rank 3 - LGU Business Permits

Team Name: Nexora

Course: IT PC - System Integration and Architecture

Instructor: Neal Errol K. Bulawan

Date: October 2026

Team Members and Roles

Member	Role
Jose Arym Diaz III	Project Lead / Architect
Princess Haneen Dibarosan	Integration Developer
Jericho M. Fegason	Data and Security Lead
Hannah L. Alcantara	QA and Documentation Lead

Executive Summary

The problem. In a typical Local Government Unit (LGU), a business owner who applies for a permit must first file at the permit office, then walk to the treasury to pay the assessed fees, and then walk back to the permit office to show the official receipt as proof of payment. The two offices run separate systems that do not talk to each other, so staff re-encode payment details by hand. This causes repeated trips, queues, encoding errors, and delays in releasing permits.

The systems. Three systems are involved: the Permit Application System (owned by the Business Permits and Licensing Office), the Treasury System (owned by the City/Municipal Treasurer's Office), and the Notification Service (owned by the LGU's IT/MIS office).

The proposed integration: PermitLink connects the three systems through three flows. (1) REST: the Permit Application System calls GET /payments/{applicationId} on the Treasury System to check payment status on demand. (2) Message broker: the Treasury System publishes payment.posted when a payment is recorded, and the Permit Application System consumes it and updates the application automatically. (3) Event: when a permit is approved, the Permit Application System publishes permit.approved, and the Notification Service consumes it and sends the applicant a notice. Only synthetic business name, owner name and payment amount cross system boundaries.

Expected benefit: Applicants no longer carry proof of payment between offices, staff no longer re-encode receipts, and applicants are told automatically when the permit is ready. Targets: payment status reflected in the permit system within 5 seconds of posting, no lost payment messages, and an approval notice sent within 10 seconds of approval.

Background and Problem

The organization: The project is modeled on a generic Philippine city or municipal LGU. Business permits are processed by the Business Permits and Licensing Office (BPLO). Fees and local taxes are collected by the Treasurer's Office. The Mayor's Office, through its IT/MIS unit, maintains shared digital services such as applicant communication. No real LGU data is used; all records are synthetic.

Current process:

1. The applicant submits a business permit application at the BPLO.
2. BPLO staff assess the fees due and give the applicant an order of payment.
3. The applicant goes to the Treasurer's Office, pays and receives an official receipt.
4. The applicant returns to the BPLO and presents the receipt as proof of payment.
5. BPLO staff manually check the receipt and re-encode the receipt number, amount and date into the permit system.
6. The application is approved and the permit is released. The applicant usually learns about it by returning to the office or calling.

Where data is siloed or re-encoded by hand:

- Payment is recorded only in the Treasury System; the Permit Application System cannot see it
- BPLO staff retype receipt details from a paper receipt, which risks typing errors and mismatched amounts.
- The applicant is the messenger between the two offices.
- Approval is not communicated unless staff or the applicant follow up manually.

Problem Statement:

Applicants go back and forth between the permit office and the treasury to show proof of payment, because the Permit Application System and the Treasury System are not integrated. This wastes applicants' time, adds manual re-encoding work and error risk for staff, and delays permit approval and release.

Objectives

General Objective: To design and demonstrate an integration of the Permit Application System, Treasury System and Notification Service that removes the need for applicants to carry proof of payment between offices, using REST, a message broker and event-based communication.

Specific Objectives:

1. Payment verification (REST): implement GET /payments/{applicationId} on the Treasury System, with 95% of requests answered within 500 ms at 50 concurrent users.
2. Automatic payment sync (broker): publish payment.posted from the Treasury System and consume it in the Permit Application System so 99% of payments are reflected within 5 seconds, with zero lost or duplicated updates in testing.
3. Approval notice (event): publish permit.approved and have the Notification Service send a notice for 100% of approved test applications within 10 seconds.
4. Data minimization: limit data crossing system boundaries to synthetic business name, owner name and payment amount (plus link identifiers), verified by a payload review with 0 extra personal fields
5. Reproducible environment: run all three systems and the broker with a single docker compose up command, starting in under 5 minutes on a standard laptop.

Scope and Limitations

In scope:

- Three integration flows: REST status check, broker-based payment.posted, and event-based permit.approved with applicant notice
- A simplified workflow: application submitted → payment pending → payment confirmed → approved → notice sent
- Integration design deliverables: API contract, message and event schemas, architecture diagrams, quality attribute test results
- Error handling: retries, duplicate messages and an unreachable Treasury System
- Service-to-service authentication and audit logging of integration events

Out of scope:

- Real LGU systems, real applicants or real payments
- Full business permit requirements (zoning, sanitary, fire safety, barangay clearance and other clearances)

- Real payment gateways, online banking and e-wallets
- Computation of actual local tax and fee schedules
- Permit renewal, amendment and closure
- Production deployment, hosting and long-term maintenance

Mocked:

- Permit Application System and Treasury System: simplified prototype services with their own databases and seeded synthetic data
- Notification channel: a mock email/SMS sink (a local mail-capture tool or a log)
- Identity and payment records: generated fake business names, owner names and amounts

Limitations: Results come from a lab environment, so performance numbers will not match a real LGU's network. The prototype covers one payment per application and one permit type. Security is shown at prototype level and is not a formal audit.

Systems to Integrate

Name	Owner	Purpose	Stack	Data held	Interface exposed
Permit Application System	BPLO	Receives applications, tracks status, checks proof of payment, approves permits	Node.js (Express), PostgreSQL, Docker	Application ID, synthetic business and owner name, assessed amount, status, payment reference	REST API; consumes payment.posted; publishes permit.approved
Treasury System	City/Municipal Treasurer's Office	Records payments of permit fees and issues official receipts	Python (FastAPI), PostgreSQL, Docker	Application ID, amount, payment date, receipt number, payment status	GET /payments/{applicationId}; publishes payment.posted
Notification Service	LGU IT/MIS Office (Mayor's Office)	Sends notices to applicants when events occur	Node.js worker, mock email/SMS sink, Docker	Notification log (event ID, recipient, message, sent time, delivery status)	Consumes permit.approved; no public API
Message	hared /	Carries	RabbitMQ,	Messages in transit	AMQP

Broker	LGU IT/MIS Office	payment.posted messages and permit.approved events reliably	Docker	only	publish/subscribe

Integration summary

#	Name	Style	From → To	Trigger	Payload
1	Payment status check	REST	Permit → Treasury	Staff or system needs to confirm payment	GET /payments/{applicationId} returns status, amount, receipt number
2	payment.posted	Message broker (durable queue, acknowledged delivery)	Treasury → Permit	A payment is recorded	applicationId, amount, receiptNo, postedAt
3	permit.approved	Event (publish/subscribe)	Permit → Notification	A permit is approved	applicationId, businessName, ownerName

Data Crossing Boundaries

Only synthetic business name, owner name, and payment amount cross system boundaries. Records are linked using the applicationId and the official receipt number, which are generated test identifiers and not personal data.

Requirements

Functional Requirements

ID	Type	Requirement
FR-1	Integration flow (REST)	The Permit Application System shall call GET /payments/{applicationId} on the Treasury System and receive the payment status (UNPAID, PAID or NOT_FOUND), amount and receipt number.
FR-2	Integration flow (Broker)	When a payment is recorded, the Treasury System shall publish payment.posted, and the Permit Application System shall consume it and mark the matching application as PAID.

FR-3	Integration flow (Event)	When a permit is approved, the Permit Application System shall publish permit.approved, and the Notification Service shall consume it and send the applicant a notice.
FR-4	User story	As a business owner, I want my payment recognized automatically by the permit office, so that I do not have to bring my receipt back and forth between offices.
FR-5	User story	As a BPLO staff member, I want to see the payment status on an application without retyping receipt details, so that I save time and avoid encoding errors.
FR-6	User story	As a treasury clerk, I want my posted payment to reach the permit system automatically, so that I do not have to issue separate proof for the applicant.
FR-7	User story	As a business owner, I want to receive a notice when my permit is approved, so that I know when to claim it without calling or visiting.
FR-8	Integration flow (Error handling)	If a message is received twice, the Permit Application System shall process it once only (idempotent handling).
FR-9	Integration flow (Error handling)	If the Treasury System is unreachable during a REST call, the Permit Application System shall return a clear "payment status unavailable" result and shall not approve the application.
FR-10	Audit	Each integration message and event shall be logged with ID, type, timestamp and outcome.

Quality attribute scenarios

ID and attribute	Source	Stimulus	Artifact	Environment	Response	Response measure
QAS-1 Performance (REST)	Permit Application System	Sends GET /payments/{applicationId}	Treasury REST API	Normal operation, 50 concurrent users	Returns payment status	95% of requests within 500 ms
QAS-2 Performance (sync latency)	Treasury clerk	Posts a payment	Broker flow to Permit System	Normal operation, 50 payments per minute	Application is updated to PAID	99% of payments reflected within 5 seconds
QAS-3	Treasury	Publishes	Broker queue	Partial failure	Message is held and	0 of 100 test messages lost

Reliability (no message loss)	System	payment.posted while the Permit System is down for 2 minutes			delivered once the Permit System restarts	during the outage
QAS-4 Reliability (idempotency)	Broker	Redelivers the same payment.posted up to 3 times	Permit Application System	Fault condition	Application is updated once only	No duplicate application updates across 100 test redeliveries
QAS-5 Performance (notification)	Permit staff	Approves a permit	Notification Service	Normal operation	Applicant notice is sent	95% of notices sent within 10 seconds of approval
QAS-6 Availability (fault isolation)	Operations	Notification Service is stopped	Permit Application System	Partial failure	Approval still completes; notice is sent after the service returns	100% of approvals succeed during the outage; queued notices delivered within 30 seconds of recovery
QAS-7 Security (access control)	Unauthorized caller	Calls the Treasury payments API without a valid service token	Treasury REST API	Normal operation	Request is rejected and logged	100% of 20 unauthenticated requests get HTTP 401 and are logged
QAS-8 Security / Privacy (data minimization)	Reviewer	Inspects all integration payloads	Messages, events and API responses	Test run	Only approved synthetic fields are present	0 payloads contain fields beyond business name, owner name, payment amount and link identifiers
QAS-9 Modifiability	Developer	Adds a second notification channel (for example SMS)	Notification Service	Development	Change is made without modifying the Permit or Treasury Systems	0 changes to Permit and Treasury code; done in no more than 1 working day
QAS-10 Deployability	Team member	Runs docker compose up on a clean laptop	Whole prototype	Development	All services start and pass a health check	All services healthy within 5 minutes

Constraits

Type	Constraint
Budget	No funding; free and open-source tools only (Docker, RabbitMQ, PostgreSQL, GitHub or GitLab). Target cost: ₱0.
Hardware	Must run on team members' laptops using Docker Compose, without dedicated servers.
Time	Follow the prefinal schedule: proposal and repository first, then design work (APIs, events, architecture).
Legal and ethical	Synthetic or anonymized data only. Real personal data would fall under the Data Privacy Act of 2012 (RA 10173), so none is collected, stored or transmitted. The design reflects data minimization and access control.
Process / policy	Support faster, simpler government transactions in line with the Ease of Doing Business and Efficient Government Service Delivery Act (RA 11032). The team will verify the exact provisions before citing them.
Technical	The integration must use the three specified styles: REST, a message broker and event-based communication.

Architecture

Chosen style: Hybrid integration. REST for the on-demand payment status check, a RabbitMQ durable queue for payment.posted, and a RabbitMQ topic exchange (publish/subscribe) for permit.approved. Each of the three services owns its own database.

Topology: Hub-and-spoke through RabbitMQ for the two asynchronous flows, plus one point-to-point REST call from the Permit System to the Treasury System. The systems never share a database.

Alternative	Strength	Weakness for this project
REST only (Treasury calls Permit directly)	Simple to build	A payment notification may be unavailable while the Permit System is down; fails no-message-loss and fault-isolation scenarios (QAS-3, QAS-6)
Shared database	Easy to report on	Breaks BPLO and Treasurer ownership; exposes extra fields (QAS-8); hard to change (QAS-9)
Webhooks	Real-time push	No durability or replay; every system must build its own retry queue
Batch file transfer	Works with old systems	Cannot meet the 5-second sync target (QAS-2); repeats today's delay
Enterprise service bus	Central routing and mapping	Too heavy for a laptop and a 5-minute start-up (QAS-10)
Apache Kafka	Strong for high-volume	More setup than needed for about 50 messages a

	streams	minute; RabbitMQ gives acknowledgments and dead-letter queues more simply
Hybrid REST + queue + topic exchange (chosen)	Each flow gets the style that fits; free; runs in one container	More moving parts; needs an outbox and idempotent consumers (ADR-003)

Why each flow gets its style

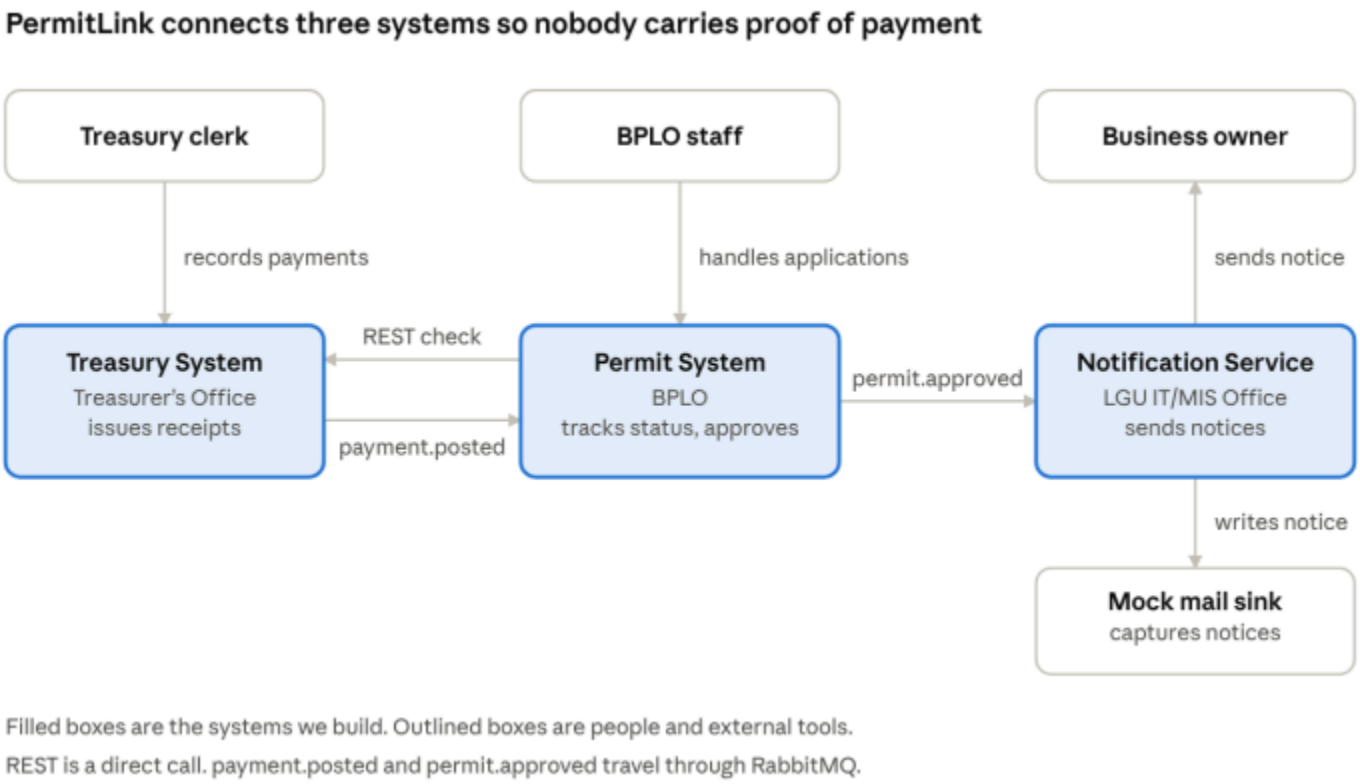
Flow	Style	Reason
Payment status check (FR-1, FR-9)	REST request/response	The caller needs a current answer right now
payment.posted (FR-2, FR-8)	Durable queue, acknowledged delivery	A payment must never be lost, even if the Permit System is down
permit.approved (FR-3)	Publish/subscribe event	Approval must not depend on notification, and new subscribers (for example SMS) can be added without touching the publisher

How the design meets the quality attribute scenarios

Scenario	Design mechanism
Performance (QAS-1, QAS-2, QAS-5)	Indexed single-row lookup; publish right after commit; one consumer per queue with a prefetch limit
Reliability (QAS-3, QAS-4)	Durable queue, persistent messages, publisher confirms, manual ack, unique messageId check
Availability (QAS-6)	Permit commits the approval first, then publishes; the notification queue holds events while the worker is down
Security (QAS-7, QAS-8)	Signed service tokens; JSON Schemas that reject extra fields
Modifiability (QAS-9)	A new channel is a new queue and handler inside the Notification Service
Deployability (QAS-10)	One docker-compose.yml with health checks

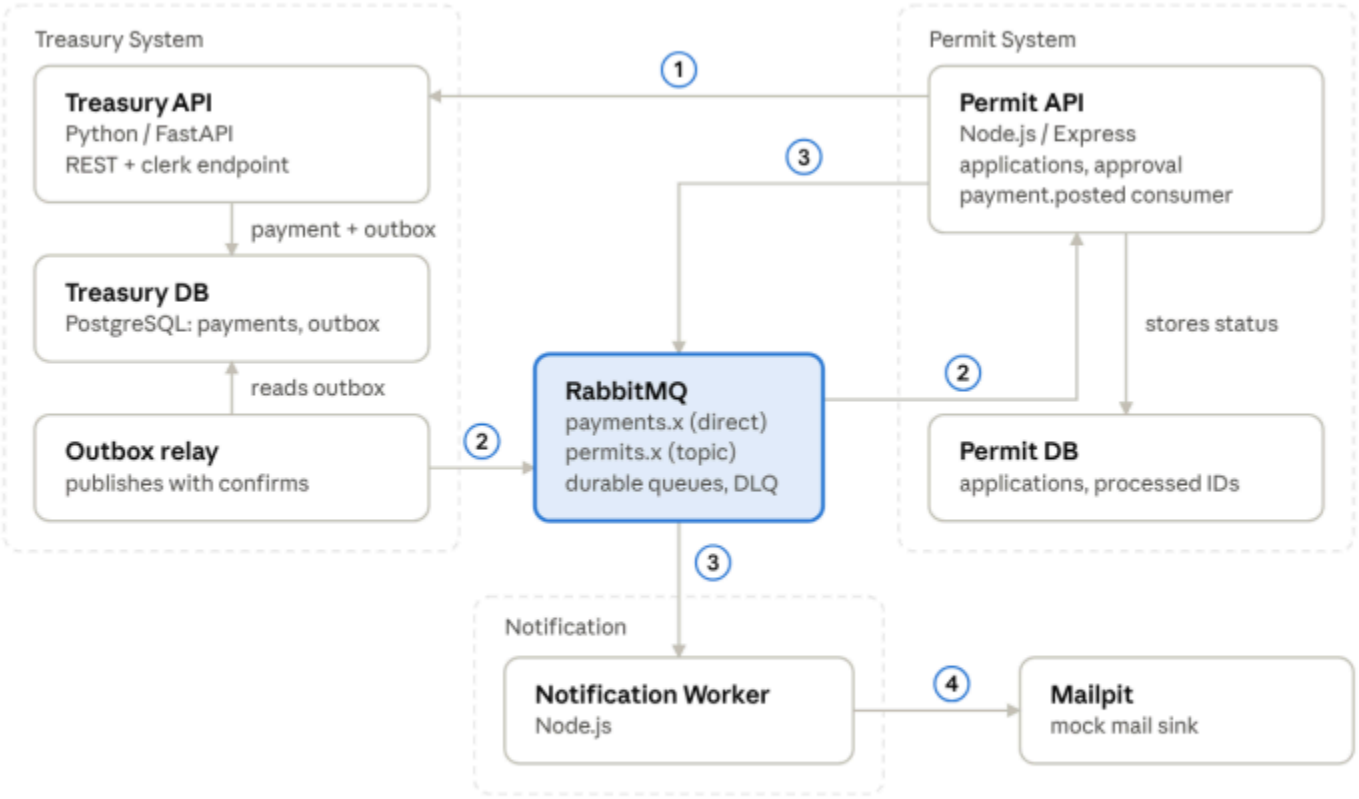
Diagrams:

- Figure 1: C4 Context diagram (three people, three systems, one mock mail sink)
- Figure 2: C4 Container diagram (containers and the four numbered flows)
- Figure 3: UML deployment diagram (Docker Compose hosts permit-service, permit-db, treasury-service, treasury-db, notification-service, rabbitmq and mailpit on one laptop)



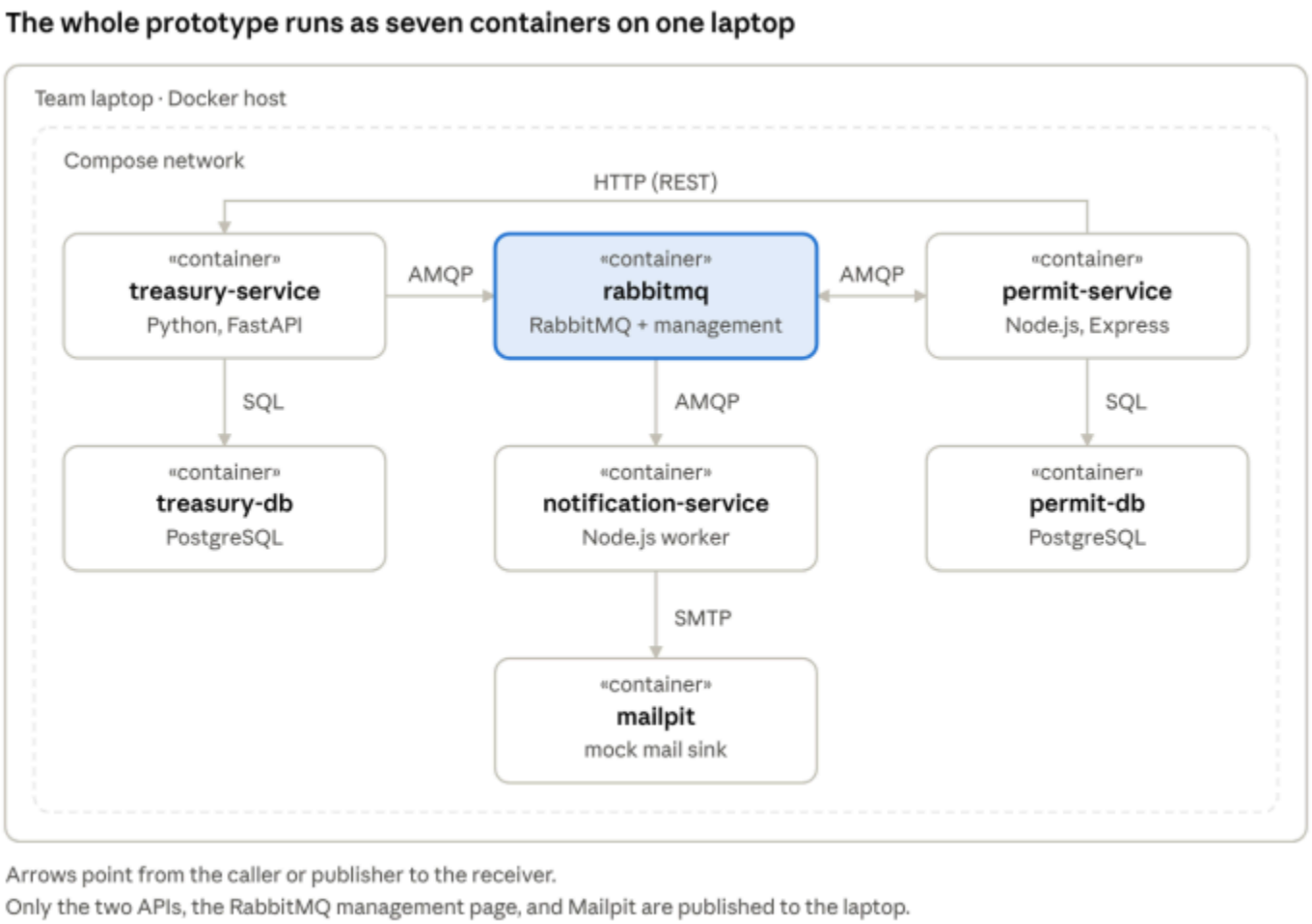
C4 Context · 3 people, 3 systems, 1 external tool

Each system keeps its own database; RabbitMQ carries the messages



1 REST: the Permit API calls GET /payments by application ID on the Treasury API
2 payment.posted: the outbox relay publishes it, the Permit API consumes it
3 permit.approved: the Permit API publishes it, the Notification Worker consumes it
4 The Notification Worker writes the notice to the mock mail sink

C4 Container · 7 containers in 3 system boundaries



UML deployment · 7 containers on one Docker host

Integration Design

OpenAPI 3 draft: /docs/openapi/treasury.yaml and /docs/openapi/permit.yaml (full files in Appendix E).

System	Method	Endpoint	Purpose
Treasury	GET	/payments/{applicationId}	Return PAID, UNPAID or NOT_FOUND with amount and receipt number (REST flow)
Treasury	POST	/payments	Treasury clerk records a payment; queues payment.posted
Treasury	GET	/health	Health check
Permit	POST	/applications	Submit a synthetic application

Permit	GET	/applications/{id}	View application and payment status
Permit	POST	/applications/{id}/approve	Approve only if payment is confirmed; publishes permit.approved
Permit	GET	/health	Health check

Every endpoint except /health needs a service or staff token. A missing or invalid token returns 401.

Message and event catalogue (draft)

Name	Style	Producer → Consumer	Exchange and queue	Payload	Delivery rule
payment.posted	Message (queue)	Treasury → Permit	payments.x, permit.payment-posted.q	applicationId, amount, receiptNo, postedAt	Persistent, publisher confirm, manual ack; 3 tries, then dead-letter queue
permit.approved	Event (pub/sub)	Permit → Notification	permits.x, notification.permit-approved.q	applicationId, businessName, ownerName	Persistent delivery with retry and dead-letter handling; duplicates skipped by eventId
payment.posted	Message (queue)	Treasury → Permit	payments.x, permit.payment-posted.q	applicationId, amount, receiptNo, postedAt	Persistent, publisher confirm, manual ack; 3 tries, then dead-letter queue
permit.approved	Event (pub/sub)	Permit → Notification	permits.x, notification.permit-approved.q	applicationId, businessName, ownerName	Persistent delivery with retry and dead-letter handling; duplicates skipped by eventId

Both carry an envelope: messageId, type, occurredAt, schemaVersion and correlationId (the applicationId). Every message and event is written to the audit log with ID, type, timestamp and outcome (FR-10).

Canonical data model

Entity	Owner	Key fields
Application	Permit System	applicationId, businessName, ownerName, assessedAmount, status, receiptNo, paidAt
Payment	Treasury System	paymentId, applicationId, amount, receiptNo, status, postedAt
ProcessedMessage	Permit System	messageId, type, processedAt, outcome
NotificationLog	Notification Service	eventId, recipient, message, sentAt, deliveryStatus
AuditLog	Each service	id, type, timestamp, outcome

Application status: SUBMITTED, PAYMENT_PENDING, PAYMENT_CONFIRMED (the PAID state in FR-2), APPROVED, NOTICE_SENT. A JSON Schema per message forbids extra fields (Appendix E).

Data mapping table (draft)

Source field	Source system	Canonical field	Rule
payments.application_id	Treasury	Application.applicationId	Match by ID; unknown ID goes to the dead-letter queue
payments.amount	Treasury	Application.paidAmount (mapped from Treasury payment amount)	Confirm only if equal to assessedAmount; otherwise log a mismatch
payments.receipt_no	Treasury	Application.receiptNo	Copied, so staff no longer retype it
payments.posted_at	Treasury	Application.paidAt	ISO 8601 UTC
business_name, owner_name	Permit	permit.approved payload	Inserted into the notice text
application_id	Permit	NotificationLog.recipient	Mock mailbox built from the ID, for example APP-000123@permitlink.test

Security and Privacy

Threat model (OWASP API Security Top 10, 2023)

Risk	Threat in this project	Mitigation
API1 Broken object-level authorization	A caller reads a payment it should not see	Role claim in the token; only permit-service and staff may call GET /payments
API2 Broken authentication	Missing, expired or forged token	Signed JWT service tokens on every call; 401 and a log entry (QAS-7)
API3 Broken object property-level authorization	Responses or messages carry extra fields	Allow-listed response fields; JSON Schemas with no extra fields (QAS-8)
API4 Unrestricted resource consumption	Flooding the API or the queue	Rate limit per token, body size limit, queue limit with dead-lettering, consumer prefetch limit
API5 Broken function-level authorization	A staff token calls POST /payments to fake a payment	POST /payments allowed only for the treasury-clerk role; others get 403
API6 Unrestricted access to sensitive business flows	Approving without payment, or replaying approval	Approve only when payment is confirmed (FR-9); approve is idempotent
API8 Security misconfiguration	Default RabbitMQ guest account, open ports	Guest disabled, management page on localhost only, one broker user per service with least privilege
API9 Improper inventory management	Old or unlisted endpoints stay exposed	OpenAPI file per service is the single inventory; contract test fails on undocumented routes
API10 Unsafe	A malformed message or response	Validate every message and REST response

consumption of APIs	is trusted	against its schema; bad ones go to the dead-letter queue

RA 10173 personal data inventory

Data	Purpose	Storage	Protection
Owner name	Address the approval notice	Permit DB	Synthetic only; sent only in permit.approved
Business name	Name the permit in the notice	Permit DB	Synthetic only; sent only in permit.approved
Payment amount and receipt number	Confirm the fee is paid	Treasury DB, Permit DB	Synthetic; sent only in payment.posted and the REST response
Application ID	Link records	All services	Generated test ID, not personal data
Audit log entries	Accountability (FR-10)	Each service	Identifiers only, no names

Not collected or sent: address, phone, email, government ID, TIN, birth date, bank or card details. The design follows the Data Privacy Act principles of transparency, legitimate purpose and proportionality. The team will verify the exact section numbers before citing them in the final paper.

Residual risk

Residual risk	Why it remains	Plan
No TLS between containers	Prototype runs on one local Docker network	Documented limitation; production would use TLS for HTTP and AMQP
Shared token signing key	A real LGU would use an identity provider	Documented limitation; key kept in .env, never in Git
No formal security audit	Out of scope (Part 5.4)	OWASP checklist review and a ZAP baseline scan only

Reliability

Failure	Behavior
Treasury down during a REST call	Permit returns "payment status unavailable", does not approve, and logs it (FR-9); staff retry later
Permit down when a payment is posted	Message waits in the durable queue and is applied when Permit restarts; none lost (QAS-3)
Notification Service down	Approval still completes; the event waits in its queue and the notice is sent after recovery (QAS-6)
Broker down	Treasury saves the payment and an outbox row in one transaction; the relay publishes once the broker returns; REST check still works
Duplicate message	Permit skips a messageId it has already processed (QAS-4, FR-8)
Invalid message	Rejected after 3 tries and moved to the dead-letter queue for

	inspection
Message delayed past 5 seconds	On approval Permit confirms status through the REST check
A service database is unavailable	Health check fails; API returns 503; the message is not acknowledged, so it is redelivered

Architecture Decision Records

ADR-001: Use RabbitMQ as the message broker

- Status: Accepted
- Context: We need durable queues, acknowledgments, dead-lettering and a topic exchange, free, in one container, for about 50 messages a minute.
- Decision: RabbitMQ with the management plugin.
- Alternatives: Kafka (heavier), webhooks (no durability), cloud queues (cost).
- Consequences: Simple setup; messages are removed after acknowledgment, so the audit log keeps the record.

ADR-002: Hybrid of REST, a work queue and a topic exchange

- Status: Accepted
- Context: One flow needs an immediate answer, one must never lose data, one must not block approval.
- Decision: REST for status, a durable queue for payment.posted, a topic exchange for permit.approved.
- Alternatives: REST only, shared database, batch files, one style for everything (Part 8).
- Consequences: Each flow meets its scenario; the team handles eventual consistency, so REST also acts as a reconciliation check.

ADR-003: Transactional outbox, publisher confirms and idempotent consumers

- Status: Accepted
- Context: Saving a payment and publishing a message are two actions, and RabbitMQ delivers at least once.
- Decision: Treasury saves the payment and an outbox row in one transaction; a relay publishes with confirms; consumers record each messageId in a unique table.
- Alternatives: Publish after saving (can lose messages), two-phase commit (not supported across PostgreSQL and RabbitMQ), exactly-once delivery (not offered).
- Consequences: One extra table and some code; gives no lost messages (QAS-3) and no duplicate updates (QAS-4).

ADR-004: One database per service with JWT service tokens

- Status: Accepted
- Context: BPLO and the Treasurer own different data; QAS-7 and QAS-8 need access control and minimal sharing.
- Decision: Separate PostgreSQL databases; services talk only through REST and the broker with short-lived signed tokens that carry a role.
- Alternatives: Shared database, static API keys, a full OAuth server.
- Consequences: Clear ownership and a smaller attack surface; token issuing is simplified (see residual risk).

ADR-005: Validate every payload with JSON Schemas

- Status: Accepted
- Context: Data minimization must be provable and bad messages must not corrupt data.
- Decision: One schema per message with no extra fields; producers validate before sending, consumers on receipt.
- Alternatives: Documentation only, a schema registry (more infrastructure).
- Consequences: Extra fields are rejected automatically (QAS-8); schema changes need a schemaVersion bump.

Technology Stack and Tools

Layer	Choice	Why
Permit Application System	Node.js with Express	Team can reuse JavaScript with the Notification Service; amqplib for AMQP
Treasury System	Python 3.12 with FastAPI	Auto-generated OpenAPI docs; fast to build
Notification Service	Node.js worker	Simple consumer with no public API
Message broker	RabbitMQ with management plugin	See ADR-001
Database	PostgreSQL, one per business service	Free; unique constraints support idempotency
Mock notification sink	Mailpit	Shows sent notices without a real provider
Packaging	Docker Compose	One command start-up (QAS-10)
Contract and docs	OpenAPI 3, Swagger UI, Mermaid or draw.io	Contract is the source of truth; diagrams stored as text
Testing	pytest, Jest, k6, Newman or	Proves the quality scenarios

	Schemathesis, OWASP ZAP	
Source control	GitHub or GitLab	Free; instructor added as member

The project targets a ₱0 software cost using free and open-source tools. Versions will be pinned in the Docker Compose and package files.

Testing Plan

Test type	Tool	What it proves
Unit	pytest, Jest	Approve only when paid, amount match, duplicate check
API and message contract	Newman or Schemathesis, schema validators	Endpoints match the OpenAPI file; messages match their schemas
Integration	Docker Compose and test scripts	All three flows work across real containers
Fault injection	docker stop and docker start	Behavior when a service or the broker is down
Performance	k6 and log timestamps	Latency targets at the stated load
Security	curl or Newman, OWASP ZAP baseline	Missing tokens rejected; role checks hold

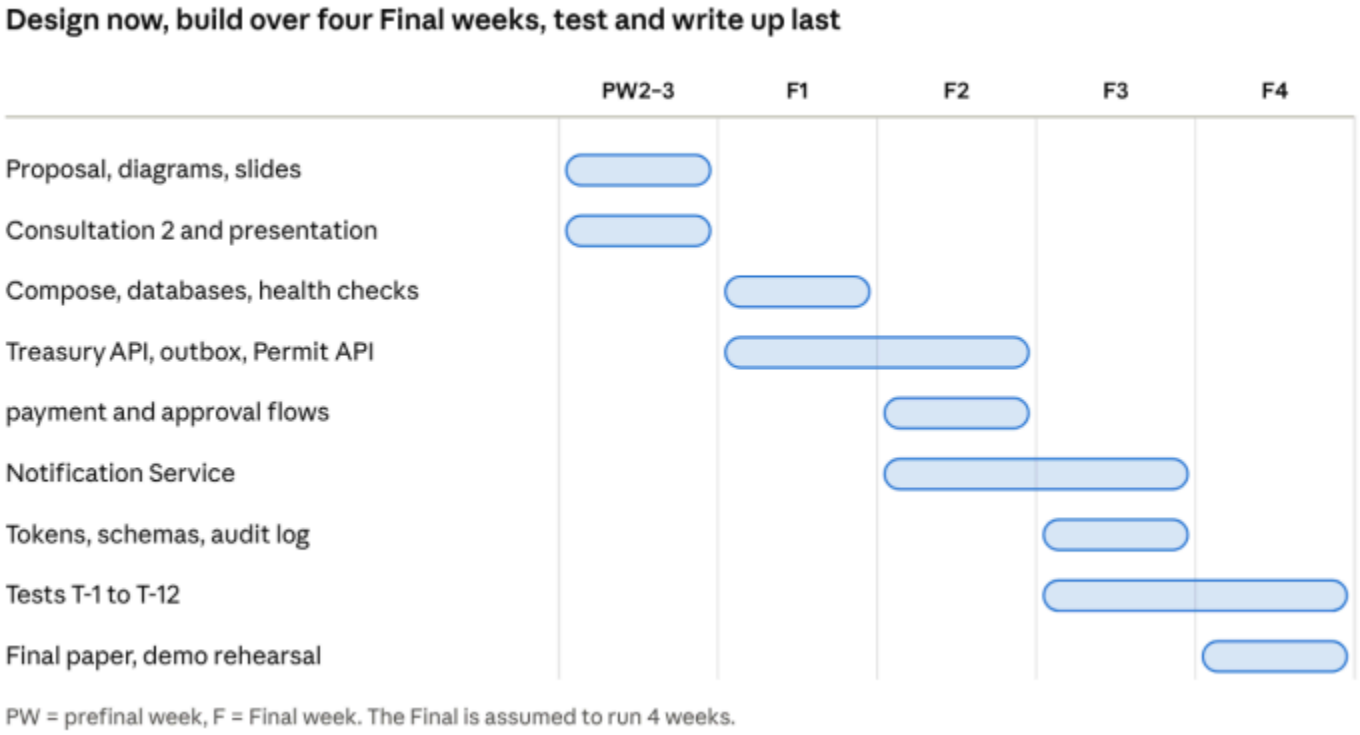
Test cases by integration flow and quality scenario

Test	Covers	Pass criterion
T-1	REST flow, QAS-1	95% of GET /payments/{applicationId} calls under 500 ms at 50 users
T-2	REST flow, FR-1	PAID, UNPAID and NOT_FOUND returned with the right fields
T-3	Broker flow, QAS-2	99% of payments reflected within 5 seconds at 50 per minute
T-4	Broker flow, QAS-3	Permit stopped 2 minutes, 100 payments posted, 0 lost
T-5	Broker flow, QAS-4	Same message redelivered 3 times, 100 messages, 0 duplicate records
T-6	Event flow, QAS-5	95% of notices sent within 10 seconds of approval; all test notices eventually delivered
T-7	Event flow, QAS-6	Notification stopped: 100% of approvals succeed; notices delivered within 30 seconds of recovery
T-8	FR-9	Treasury stopped: "payment status unavailable" and no approval
T-9	QAS-7, OWASP API1, API5	20 of 20 unauthenticated calls get 401 and are logged; staff token on POST /payments gets 403
T-10	QAS-8, FR-10	0 payloads with extra fields; every message has an audit entry
T-11	QAS-9	A second channel is added with 0 changes to Permit and Treasury code, in 1 working day
T-12	QAS-10	docker compose up on a clean laptop: all services healthy within 5 minutes

Test data comes from seed scripts that generate fake names and amounts. Results are saved in /docs/test-results.

Work Plan (Gantt Chart)

Phase	Week	Work
Design	Prefinal Weeks 2–3	Parts 1–18, diagrams, API and event drafts, slides, Consultation 2
Build 1	Final, week 1	Docker Compose with health checks, databases, Treasury API with outbox, Permit API
Build 2	Final, week 2	payment.posted flow, permit.approved flow, Notification Service
Build 3	Final, week 3	Service tokens, broker permissions, schema validation, audit log; start tests
Wrap-up	Final, week 4	Tests T-1 to T-12, final paper, demo rehearsal



Gantt chart · 9 tasks over PW2–3 and 4 Final weeks

The Final project phase is planned for four weeks. Replace week labels with calendar dates once the instructor confirms the schedule.

Responsibility Matrix

Task	Lead / Architect	Integration Dev	Data and Security	QA and Docs

Architecture diagrams and ADRs	R	C	C	I
OpenAPI file, event catalogue, schemas	A	R	C	C
Treasury, Permit and Notification services	A	R	C	I
Docker Compose and repository	R	C	I	A
Threat model, RA 10173 inventory, tokens	C	C	R	I
Failure handling tests (T-4 to T-8)	C	R	C	A
Testing plan and test results	C	C	C	R
Proposal, slides and minutes	C	C	C	R

R = Responsible, A = Accountable, C = Consulted, I = Informed. Roles: Jose Arym Diaz III (Project Lead / Architect), Princess Haneen Dibarosan (Integration Developer), Jericho M. Fegason (Data and Security Lead), Hannah L. Alcantara (QA and Documentation Lead). Git history should reflect this split.

Risks (minimum 5)

#	Risk	Likelihood	Impact	Mitigation
1	Work is underestimated (three services plus a broker)	Medium	High	Build the thinnest path first: REST, then payment.posted, then permit.approved; weekly check against the plan
2	Docker Compose does not start cleanly on every laptop	Medium	High	Pin versions, add health checks, test on every laptop in Build 1
3	Lost or duplicate messages from bugs	Medium	High	Outbox, publisher confirms, manual ack, unique messageId; run T-4 and T-5 early
4	Performance targets missed on laptops	Low	Medium	Index the lookup, limit prefetch, run k6 early; report measured values if missed
5	Real personal data enters the project	Low	High	Seed scripts generate fake data only; payload review (T-10); review before each commit
6	A member is absent or unreachable	Medium	Medium	Named backup per role; project board; action items in minutes
7	Scope creep (more permits, real gateway)	Medium	Medium	Out-of-scope list in Part 5; new ideas go to a later list

Executive Summary

Applicants at a Philippine LGU walk between the Business Permits and Licensing Office and the Treasurer's Office to carry proof of payment, because the Permit Application System and the Treasury System do not talk to each other, and staff retype receipts by hand. PermitLink connects the Permit Application System,

the Treasury System and the Notification Service with a hybrid design: a REST call for payment status, a durable RabbitMQ queue for payment.posted, and a topic exchange for permit.approved. A transactional outbox, publisher confirms and idempotent consumers prevent lost and duplicate payments, while service tokens, least-privilege broker users and schema validation protect access and data. Ten measurable scenarios and twelve tests will show payment reflected within 5 seconds for 99% of payments, 0 of 100 messages lost, notices within 10 seconds, and a full start-up in under 5 minutes with one command. Only synthetic data is used, so applicants stop carrying receipts and staff stop retyping them.

References and Appendices

References (APA 7)

- Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.
- Brown, S. (n.d.). *The C4 model for visualising software architecture*. https://c4model.com
- Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns*. Addison-Wesley.
- OpenAPI Initiative. (2021). *OpenAPI specification, version 3.0.3*. https://spec.openapis.org/oas/v3.0.3
- OWASP Foundation. (2023). *OWASP API security top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- RabbitMQ. (n.d.). *RabbitMQ documentation*. https://www.rabbitmq.com/docs
- Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Republic of the Philippines.
- Republic Act No. 11032. (2018). *Ease of Doing Business and Efficient Government Service Delivery Act of 2018*. Republic of the Philippines.

Appendices (not counted in the 12–20 page limit)

Appendix A: Research Tools Used

The team used Google Scholar and Google Search to find related studies, references, and supporting information for the PermitLink Prefinal Proposal.

Appendix B: Consultation 1 Minutes

- Date: October 30, 2026 (Saturday)
- Time: 9:00 AM – 10:30 AM
- Details: Minutes of the first consultation meeting with the instructor, including the discussions, feedback, and recommendations regarding the project proposal.

Appendix C: Approved Scenario Record and Candidate Scenarios

- Approved Scenario: Rank 3 – LGU Business Permits
- Other candidate scenarios considered by the team.

Appendix D: Team Roles and Responsibilities

- Diaz III – Pitching deck and Figma presentation.
- Fegason – PermitLink Prefinal Proposal paper.
- Dibarosan – PermitLink Prefinal Proposal paper.
- Alcantara – PermitLink Prefinal Proposal paper.

Appendix E: Full OpenAPI Files and JSON Schemas

- Complete OpenAPI specification files and JSON Schemas are available in the project repository under the /docs folder.

Appendix F: Revision Table

- Include this appendix only if the proposal receives an Approved with Revisions verdict. It should contain the instructor's comments, the revisions made, and the corresponding sections updated.